

```
# Set up Kaggle API and download dataset using kaggle.json credentials
```

```
!pip install kaggle
!mkdir -p ~/.kaggle
```

```
from google.colab import files
files.upload()
```

```
!cp kaggle.json ~/.kaggle/
```

```
!chmod 600 ~/.kaggle/kaggle.json
```

```
!kaggle datasets download -d carrie1/ecommerce-data
!unzip ecommerce-data.zip
```

```
Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages (1.7.4.5)
Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages (from kaggle) (6.3.0)
Requirement already satisfied: certifi>=14.05.14 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2026.1.4)
Requirement already satisfied: charset-normalizer in /usr/local/lib/python3.12/dist-packages (from kaggle) (3.4.4)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from kaggle) (3.11)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kaggle) (5.29.5)
Requirement already satisfied: python-dateutil>=2.5.3 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.9.0.post0)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.32.4)
Requirement already satisfied: setuptools>=21.0.0 in /usr/local/lib/python3.12/dist-packages (from kaggle) (75.2.0)
Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.17.0)
Requirement already satisfied: text-unidecode in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kaggle) (4.67.1)
Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.5.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from kaggle) (0.5.1)
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving kaggle.json to kaggle.json

Dataset URL: <https://www.kaggle.com/datasets/carrie1/ecommerce-data>

License(s): unknown

Downloading ecommerce-data.zip to /content

0% 0.00/7.20M [00:00<?, ?B/s]

100% 7.20M/7.20M [00:00<00:00, 697MB/s]

Archive: ecommerce-data.zip

```
# Import required libraries
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report, r2_score
```

```
# Read the CSV file with Latin-1 encoding to prevent errors
```

```
df = pd.read_csv("data.csv", encoding='latin-1')
```

▼ Data Overview

```
# Explore the dataset
```

```
print('First five rows of the dataset:\n', df.head())
print('Last five rows of the dataset:\n', df.head())
```

First five rows of the dataset:

	InvoiceNo	StockCode	Description	Quantity	\
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	
1	536365	71053	WHITE METAL LANTERN	6	
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	

```
InvoiceDate  UnitPrice  CustomerID      Country
0 12/1/2010 8:26      2.55    17850.0  United Kingdom
1 12/1/2010 8:26      3.39    17850.0  United Kingdom
2 12/1/2010 8:26      2.75    17850.0  United Kingdom
3 12/1/2010 8:26      3.39    17850.0  United Kingdom
4 12/1/2010 8:26      3.39    17850.0  United Kingdom

Last five rows of the dataset:
InvoiceNo  StockCode      Description  Quantity \
0    536365    85123A  WHITE HANGING HEART T-LIGHT HOLDER      6
1    536365    71053                WHITE METAL LANTERN      6
2    536365    84406B      CREAM CUPID HEARTS COAT HANGER      8
3    536365    84029G  KNITTED UNION FLAG HOT WATER BOTTLE      6
4    536365    84029E      RED WOOLLY HOTTIE WHITE HEART.      6

InvoiceDate  UnitPrice  CustomerID      Country
0 12/1/2010 8:26      2.55    17850.0  United Kingdom
1 12/1/2010 8:26      3.39    17850.0  United Kingdom
2 12/1/2010 8:26      2.75    17850.0  United Kingdom
3 12/1/2010 8:26      3.39    17850.0  United Kingdom
4 12/1/2010 8:26      3.39    17850.0  United Kingdom
```

```
# Total no. of rows and columns of the dataset
df.shape
```

(541909, 8)

```
# Retrieve the columns names
```

```
df.columns
```

```
Index(['InvoiceNo', 'StockCode', 'Description', 'Quantity', 'InvoiceDate',
       'UnitPrice', 'CustomerID', 'Country'],
      dtype='object')
```

```
# Statistical Measures of the dataset
df.describe()
```

	Quantity	UnitPrice	CustomerID
count	541909.000000	541909.000000	406829.000000
mean	9.552250	4.611114	15287.690570
std	218.081158	96.759853	1713.600303
min	-80995.000000	-11062.060000	12346.000000
25%	1.000000	1.250000	13953.000000
50%	3.000000	2.080000	15152.000000
75%	10.000000	4.130000	16791.000000
max	80995.000000	38970.000000	18287.000000

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   InvoiceNo    541909 non-null object
1   StockCode    541909 non-null object
2   Description  540455 non-null object
3   Quantity     541909 non-null int64
4   InvoiceDate  541909 non-null object
5   UnitPrice    541909 non-null float64
6   CustomerID   406829 non-null float64
7   Country      541909 non-null object
dtypes: float64(2), int64(1), object(5)
memory usage: 33.1+ MB
```

Exploratory Data Analysis (EDA)

Duplicate & Missing values detection

```
# Missing values in the dataset
missing = df.isna().sum().sort_values(ascending=False)
print(((missing/len(df)) * 100).round(2))
```

```
CustomerID      25.16
StockCode       0.00
InvoiceNo       0.00
Description      0.00
Quantity        0.00
InvoiceDate     0.00
UnitPrice       0.00
Country         0.00
IsCancellation  0.00
InvoiceMonth    0.00
InvoiceHour     0.00
dtype: float64
```

```
# Fill up the values with the appropriate values
df['Description'] = df['Description'].fillna('')

# Keep missing CustomerID as NaN
df['CustomerID'] = df['CustomerID'].astype('Int64')
```

```
# Find duplicated values in the dataset
df.duplicated(
    subset=['InvoiceNo', 'StockCode', 'Quantity', 'InvoiceDate', 'UnitPrice']
).sum()
```

```
np.int64(5270)
```

```
# Inspection of duplicated transaction lines
```

```
dupes = df[df.duplicated(
    subset=['InvoiceNo', 'StockCode', 'Quantity', 'InvoiceDate', 'UnitPrice'], keep= False
)]
dupes.sort_values(['InvoiceNo', 'StockCode', 'InvoiceDate', 'UnitPrice'])
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
494	536409	21866	UNION JACK FLAG LUGGAGE TAG	1	12/1/2010 11:45	1.25	17908	United Kingdom
517	536409	21866	UNION JACK FLAG LUGGAGE TAG	1	12/1/2010 11:45	1.25	17908	United Kingdom
485	536409	22111	SCOTTIE DOG HOT WATER BOTTLE	1	12/1/2010 11:45	4.95	17908	United Kingdom
539	536409	22111	SCOTTIE DOG HOT WATER BOTTLE	1	12/1/2010 11:45	4.95	17908	United Kingdom
489	536409	22866	HAND WARMER SCOTTY DOG DESIGN	1	12/1/2010 11:45	2.10	17908	United Kingdom
...	...	...	...	...	...	...	...	...
440149	C574510	22360	GLASS JAR ENGLISH CONFECTIONERY	-1	11/4/2011 13:25	2.95	15110	United Kingdom
461407	C575940	23309	SET OF 60 I LOVE LONDON CAKE CASES	-24	11/13/2011 11:38	0.55	17838	United Kingdom

```
# Drop Duplicated Transaction lines
```

```
df.drop_duplicates(subset=['InvoiceNo', 'StockCode', 'Quantity', 'InvoiceDate', 'UnitPrice'], keep= 'first', inplace=True)
df.sort_values(['InvoiceNo', 'StockCode', 'Quantity', 'InvoiceDate', 'UnitPrice'])
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
6	536365	21730	GLASS STAR FROSTED T-LIGHT HOLDER	6	12/1/2010 8:26	4.25	17850	United Kingdom
5	536365	22752	SET 7 BABUSHKA NESTING BOXES	2	12/1/2010 8:26	7.65	17850	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	12/1/2010 8:26	3.39	17850	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/2010 8:26	3.39	17850	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/2010 8:26	3.39	17850	United Kingdom
...	...	...	...	...	...	...	...	...
540449	C581490	23144	ZINC T-LIGHT HOLDER STARS SMALL	-11	12/9/2011 9:57	0.83	14397	United Kingdom
541541	C581499	M	Manual	-1	12/9/2011 10:28	224.69	15498	United Kingdom

Cancellation & Returns overview

```
# Identify cancellations and Returns in the dataset
df['IsCancellation'] = df['InvoiceNo'].str.startswith('C')
df['IsCancellation'].value_counts()
```

	count
IsCancellation	
False	527388
True	9251

dtype: int64

```
returns = (df['Quantity'] < 0).value_counts()
print((returns/ len(df['Quantity'])* 100).round(2))
```

Quantity	
False	98.03
True	1.97
Name: count, dtype: float64	

Product Analysis

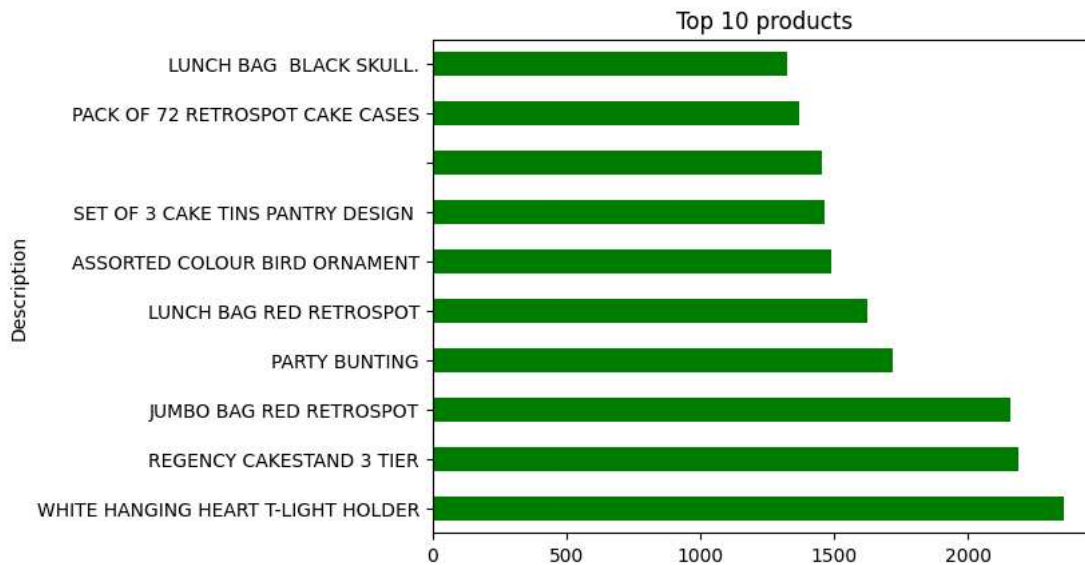
```
# Most frequently purchased products by count
df['Description'].value_counts().head(10)
```

	count
Description	
WHITE HANGING HEART T-LIGHT HOLDER	2357
REGENCY CAKESTAND 3 TIER	2189
JUMBO BAG RED RETROSPOT	2156
PARTY BUNTING	1720
LUNCH BAG RED RETROSPOT	1625
ASSORTED COLOUR BIRD ORNAMENT	1488
SET OF 3 CAKE TINS PANTRY DESIGN	1465
	1454
PACK OF 72 RETROSPOT CAKE CASES	1367
LUNCH BAG BLACK SKULL.	1323

dtype: int64

```
# Bar chart of Top 10 frequently purchased products
df['Description'].value_counts().head(10).plot(kind = 'barh', color = 'green', title= 'Top 10 products')
```

```
<Axes: title={'center': 'Top 10 products'}, ylabel='Description'>
```



```
# Most frequently returned products by count
df[df['Quantity'] < 0]['Description'].value_counts().head(10)
```

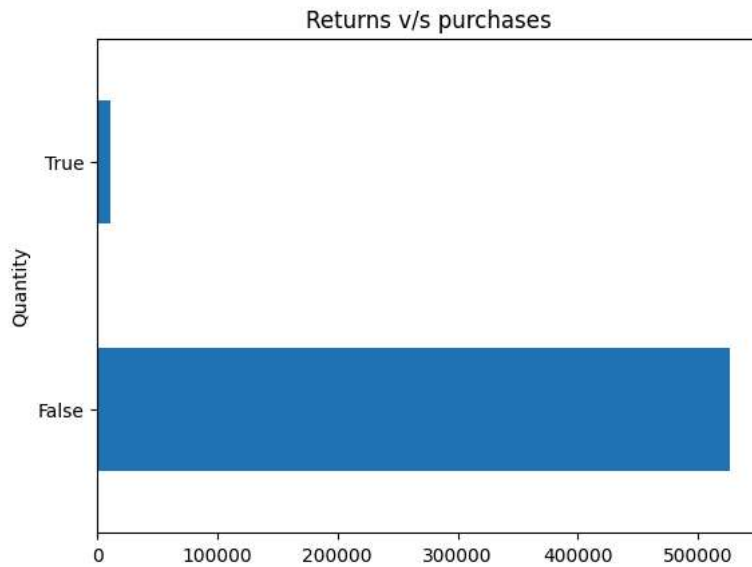
	count
Description	
	862
Manual	244
REGENCY CAKESTAND 3 TIER	180
POSTAGE	126
check	120
JAM MAKING SET WITH JARS	87
Discount	77
SET OF 3 CAKE TINS PANTRY DESIGN	73
SAMPLES	60
ROSES REGENCY TEACUP AND SAUCER	54

dtype: int64

```
# Bar chart of returns v/s normal purchases
```

```
df['Quantity'].lt(0).value_counts().plot(kind = 'barh', title = 'Returns v/s purchases')
```

<Axes: title={'center': 'Returns v/s purchases'}, ylabel='Quantity'>



Transactions & Time-based trends

```
# Total transactions per customer
df.groupby('CustomerID')['InvoiceNo'].nunique().sort_values(ascending = False).head(10)
```

InvoiceNo	
CustomerID	
14911	248
12748	224
17841	169
14606	128
15311	118
13089	118
12971	89
14527	86
13408	81
14646	77

dtype: int64

```
# Convert InvoiceDate to Date-time
```

```
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])
```

```
# Sorting out the Monthly trends
```

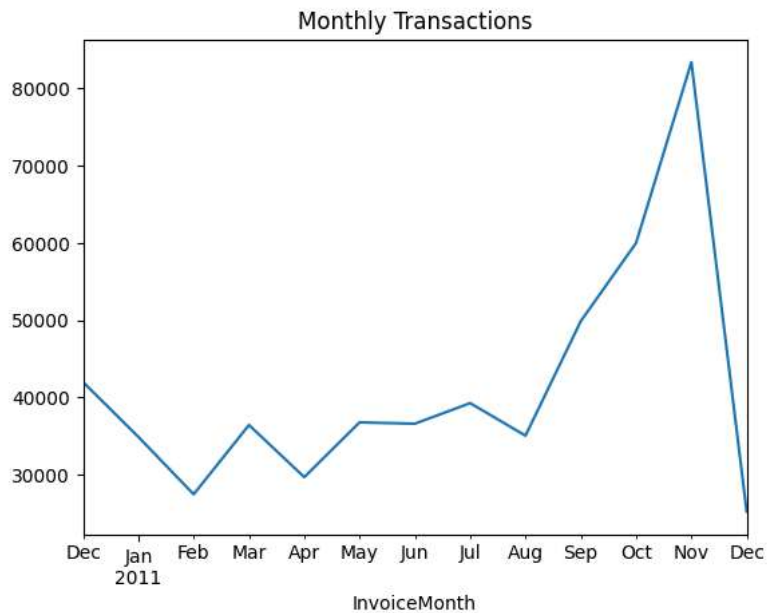
```
df['InvoiceMonth'] = df['InvoiceDate'].dt.to_period('M')
df['InvoiceMonth'].value_counts().sort_index()
```

InvoiceMonth	count
2010-12	41981
2011-01	34900
2011-02	27479
2011-03	36439
2011-04	29701
2011-05	36781
2011-06	36609
2011-07	39267
2011-08	35064
2011-09	49861
2011-10	59969
2011-11	83342
2011-12	25246

dtype: int64

```
df['InvoiceMonth'].value_counts().sort_index().plot(kind='line', title='Monthly Transactions')
```

<Axes: title={'center': 'Monthly Transactions'}, xlabel='InvoiceMonth'>



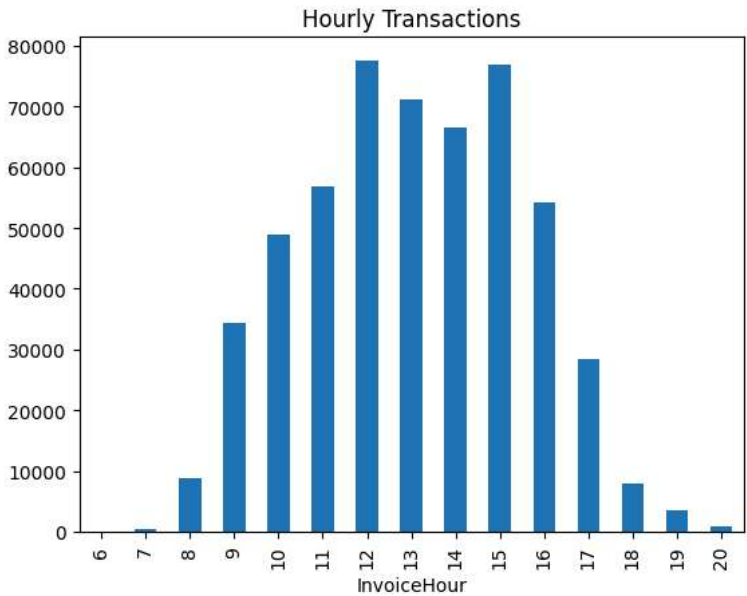
```
# Counting the Hourly trends
df['InvoiceHour'] = df['InvoiceDate'].dt.hour
df['InvoiceHour'].value_counts().sort_index()
```

InvoiceHour	count
6	41
7	383
8	8906
9	34314
10	48808
11	56948
12	77573
13	71246
14	66572
15	76938
16	54134
17	28371
18	7941
19	3617
20	847

dtype: int64

```
df['InvoiceHour'].value_counts().sort_index().plot(kind='bar', title='Hourly Transactions')
```

<Axes: title={'center': 'Hourly Transactions'}, xlabel='InvoiceHour'>



Customer & Geography Overview

```
# Total no. of customers
df['CustomerID'].nunique()
```

4372

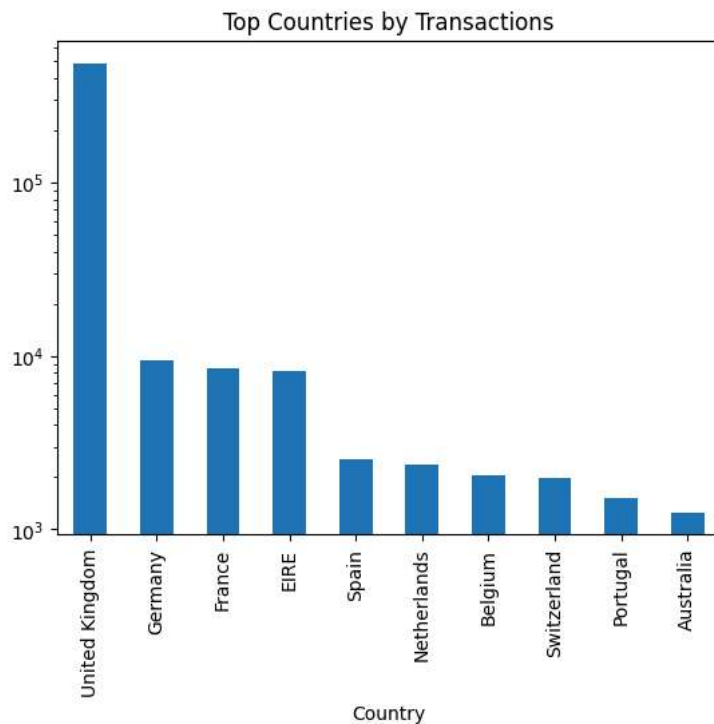
```
# Retrieve the Top 10 countries by transaction count
df['Country'].value_counts().head(10)
```

	count
Country	
United Kingdom	490298
Germany	9480
France	8541
EIRE	8184
Spain	2528
Netherlands	2371
Belgium	2069
Switzerland	1994
Portugal	1510
Australia	1258

dtype: int64

```
df['Country'].value_counts().head(10).plot(kind='bar', title='Top Countries by Transactions',logy=True)
```

<Axes: title={'center': 'Top Countries by Transactions'}, xlabel='Country'>



RFM

```
df_model = df.copy()
```

```
# Remove cancellations, returns, and orders without customer id
```

```
df_model = df_model[~df_model['InvoiceNo'].str.startswith('C')]
```

```
df_model = df_model[df_model['Quantity'] > 0]
```

```
df_model = df_model.dropna(subset=['CustomerID'])
```

```
# Convert InvoiceDate to Date-time
```

```
df_model['InvoiceDate'] = pd.to_datetime(df_model['InvoiceDate'], errors='coerce')
```

```
# Drop Duplicated Rows
df_model = df_model.drop_duplicates(
    subset=['InvoiceNo', 'StockCode', 'Quantity', 'InvoiceDate', 'UnitPrice']
)
```

```
df_model.head()
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	IsCancellation	InvoiceMonth	InvoiceYear
0	536365	85123A	WHITE HANGING HEART T- LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850	United Kingdom	False	2010-12	2010
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850	United Kingdom	False	2010-12	2010
2	536365	84406B	CREAM CUPID HEARTS COAT	8	2010-12-01 08:26:00	2.75	17850	United Kingdom	False	2010-12	2010

```
# Total Monetary Value of each customer

df_model['TotalPrice'] = df_model['Quantity'] * df_model['UnitPrice']

# Snapshot date

snapshot_date = df_model['InvoiceDate'].max() + pd.Timedelta(days=1)

# RFM analysis

rfm = df_model.groupby('CustomerID').agg({
    'InvoiceDate' : lambda x: (snapshot_date - x.max()).days,
    'InvoiceNo' : 'nunique',
    'TotalPrice' : 'sum'
}).reset_index()

rfm.columns = ['CustomerID', 'Recency', 'Frequency', 'Monetary']
```

```
# RFM scores

rfm['R_Score'] = pd.qcut(rfm['Recency'], 4, labels=[4,3,2,1])

rfm['F_Score'] = (
    pd.qcut(rfm['Frequency'], 4, duplicates='drop')
    .cat.codes + 1
)

rfm['M_Score'] = (
    pd.qcut(rfm['Monetary'], 4, duplicates='drop')
    .cat.codes + 1
)

rfm['RFM_Score'] = rfm[['R_Score', 'F_Score', 'M_Score']].astype(int).sum(axis=1)
```

```
rfm.head()
```

	CustomerID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Score
0	12346	326	1	77183.60	1	1	4	6
1	12347	2	7	4310.00	4	3	4	11
2	12348	75	4	1797.24	2	2	4	8
3	12349	19	1	1757.55	3	1	4	8
4	12350	310	1	334.40	1	1	2	4

```
# RFM segmentation
```

```
def segment_customers(row):
    if row['RFM_Score'] >= 10:
        return 'Champions'
    elif row['RFM_Score'] >= 8:
        return 'Loyal'
    elif row['RFM_Score'] >= 6:
        return 'Potential'
    elif row['RFM_Score'] >= 4:
        return 'At risk'
    else:
        return 'Lost'
```

```
rfm['Segment'] = rfm.apply(segment_customers, axis=1)
rfm['Segment'].value_counts()
```

```
count
Segment
At risk    1151
Potential  1062
Loyal      829
Champions  796
Lost       501
```

```
dtype: int64
```

```
rfm.groupby('Segment')[['Recency', 'Frequency', 'Monetary']].mean()
```

```
Recency  Frequency  Monetary
Segment
At risk    140.100782    1.422242    385.258906
Champions   12.868090   12.790201   7228.356319
Lost        261.642715    1.121756   169.279541
Loyal       36.034982    4.463209   1917.038783
Potential   64.954802    2.312618   956.137676
```

```
segment_clv = rfm.groupby('Segment')['Monetary'].mean().round(2)
print(segment_clv)
```

```
Segment
At risk    385.26
Champions   7228.36
Lost        169.28
Loyal      1917.04
Potential   956.14
Name: Monetary, dtype: float64
```

Machine Learning

```
# Supervised ML classification algorithm
```

```
X = rfm[['Recency', 'Frequency', 'Monetary']]
y = rfm['Segment']
```

```
# Encode labels using Labelencoder
encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)
```

```
# Train and Test the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size= 0.2, random_state = 42)
```

```
# RandomForest Classification Model
model = RandomForestClassifier(random_state = 42)
```

